Regex drill: "give a regular expression for this"

Sixteen questions in the mock's format (alphabet, description, must-match and must-not examples), each with the parts cut, the expression, the checks and the trap

Compilers

2026-11-02

What the mock asked. Mock 1 had two regex questions: 1.7(b) *numeric literals* ("give a regular expression; say which of these match and why the others do not") and 2.3(a) *comments* over a reduced alphabet $\{/, *, x\}$, followed by its DFA and the question whether nested comments are regular. Mock 2 tested the scanner (longest match with backup), not regex writing. No question asked for a JFlex/flex specification: flex appears only in the answers, as the behaviour behind rule order (longest match, then the first rule) and nested comments (a start condition plus a depth counter).

The method every time (§2 of the methods sheet). Cut the lexeme into parts; mark each part mandatory, optional `?`, or repeated `*`/`+`; write one sub-expression per part with | inside a part; then split every must-match example into the parts and, for every must-not example, name the part that rejects it. D stands for `[0-9]`, L for `[a-zA-Z]`. Write x for "any other character" when the question reduces the alphabet.

Method: a body that must stop at the first closer cd

For comments, strings and "does not contain w" questions. Track one thing while reading the body: *did I just read a run of c?* Two states, N (last symbol is not c) and C (just read one or more c). From N any symbol other than c stays in N and a c goes to C. From C another c stays, a d is forbidden (it would close), anything else returns to N. Read the regex off the loops at N:

$$\text{body} = (\ [\text{not } c]\ \mid\ c^{+}\,[\text{neither } c \text{ nor } d]\ )^{*}, \qquad \text{closer} = c^{+}\,d.$$

Every body item ends in a non-c symbol, so the body can never end in a run of c; the closer's c^{+} takes that run (`**/`, `(***)`). A lone d is harmless and belongs in "not c". With no closer ("does not contain bb"), add the trailing run by hand: $(a \mid ba)^{*}b?$. A three-symbol closer $c\,c\,d$ needs one more state, see §11.

1 Integers with digit separators

Practice

Given. Alphabet $D \cup \{_\}$. An integer literal is one or more digits, where groups of digits may be separated by single underscores, as in Java/Python.

Your task. (a) Write the regex. (b) Show the matches of `7`, `1_000`, `12_34_5`. (c) Explain why `_1`, `1_`, `1__0` are not matched.

Solution

Parts. A first group D^+ (mandatory), then zero or more of "underscore then group".

$$D^+ \, (\, _ \, D^+ \,)^*$$

Checks. `7`: one group. `1_000`: group `1`, then `_000`. `12_34_5`: group `12`, then `_34`, then `_5`. `_1`: the first part is D^+, mandatory, and `_` is not a digit. `1_`: the repeated part is $_D^+$, so an underscore must be followed by a digit. `1__0`: same part; after the first `_` comes another `_`, not a digit.

Gotcha

$(D \mid _)^+$ or $D^+(_^*D^+)^*$: the first accepts `__`, the second accepts `1__0`. "Single underscore between digits" means the underscore lives *inside* the repeated part, never at either end.

2 Floating-point literals, the other convention

Practice

Given. Alphabet $D \cup \{.\}$. A float has an optional integer part and an optional fraction part, but **at least one digit somewhere**: `3.`, `.5`, `3.5` and `3` are all floats (unlike mock 1.7(b), where the integer part was mandatory).

Your task. (a) Write the regex. (b) Show the matches of `3`, `3.`, `.5`, `3.5`. (c) Explain why `.`, `3..5`, `3.5.` are not matched.

Solution

Parts. Two shapes, so two alternatives: either the integer part is present (then the fraction is optional and may be empty after the dot), or it is absent (then the dot and at least one digit are mandatory).

$$D^+ \, (\, . \, D^* \,)? \quad \mid \quad . \, D^+$$

Checks. `3`: left alternative, no dot. `3.`: left, dot with $D^* = \varepsilon$. `.5`: right alternative. `3.5`: left. `.`: the right alternative needs D^+ after the dot, the left needs D^+ before. `3..5`: after `3.` the regex allows only digits; a second dot has no part to go to. `3.5.`: same, the dot appears only once in either alternative.

Gotcha

$D^*.?D^*$ matches the empty string and the lone dot `.`: "at least one digit" is the whole difficulty and must be enforced by the shape of the alternatives, not left to D^*.

3 Identifiers with at least one letter

Practice

Given. Alphabet $\{a, _, 1\}$ (a stands for any letter). An identifier starts with a letter or an underscore, continues with letters, underscores and digits, and must contain **at least one letter** (so `_` and `__1` are not identifiers).

Your task. (a) Write the regex. (b) Show the matches of `a`, `_a1`, `__a`. (c) Explain why `1a`, `_`, `_1` are not matched.

Solution

Parts. Everything before the first letter can only be underscores (no digit may come before the first letter, since the first symbol is a letter or an underscore and an underscore run cannot contain a digit). Then the first letter, mandatory. Then anything.

$$_^*\ a\ (\,a \mid _ \mid 1\,)^*$$

Checks. `a`: zero underscores, the letter, nothing. `_a1`: one underscore, letter, then `1`. `__a`: two underscores, letter.
`1a`: the prefix part is $_^*$, a digit cannot precede the first letter. `_`: the mandatory letter is missing. `_1`: after the underscores a letter must come, not a digit.

Gotcha

"At least one letter" is a *global* constraint, and regexes have no "somewhere": you encode it by anchoring on the **first** letter and describing what may come before it. The same move gives "contains `ab`": $(a \mid b)^*\, ab\, (a \mid b)^*$.

4 SQL string literals with doubled quotes

Practice

Given. Alphabet $\{', x\}$. A string literal is enclosed in single quotes; inside, a quote is written as two consecutive quotes (`'it"s'` is the string `it's`). There is no backslash.

Your task. (a) Write the regex. (b) Show the matches of `"`, `'x'`, `'it"s'` (letters are x). (c) Explain why `'it's'` and `"'` are not matched as one literal, and what a longest-match scanner returns on each.

Solution

Parts. Opening quote; body items, each either x or the pair `"`; closing quote.

$$\texttt{'}\ (x \mid \texttt{"})^*\ \texttt{'}$$

Checks. `"`: zero items. `'x'`: one item. `'it"s'`: items x, x, the pair `"`, x, then the closing quote. `'it's'`: after `'it` the next symbol is `'` followed by s, which is not the pair, so that quote closes the literal at `'it'`; the leftover `s'` is not part of it. A longest-match scanner returns the token `'it'`, then scans `s` and `'` separately (the final lone quote is a lexical error or the start of a new literal). `"'`: after the opening quote, `"` is one body item, and then the closing quote is missing; the scanner backs up to the last accepting position, which is after `"`, and returns the empty literal `"`, leaving one `'`.

Gotcha

`'`$(x \mid$ `'`$)^*$`'` ("anything between quotes") accepts `'it's'` and, worse, with longest match it swallows two literals and the code between them into one token: the mock's 2.3(a) trap in a different costume.

5 Clock times

Practice

Given. Alphabet $D \cup \{:\}$. A time is `HH:MM` on a 24-hour clock, always with two digits per field: `00:00` to `23:59`.

Your task. (a) Write the regex. (b) Show the matches of `09:30`, `23:59`. (c) Explain why `24:00`, `9:30`, `12:60` are not matched.

Solution

Parts. The hour has two cases, split on the first digit: first digit `0` or `1` with any second digit, or first digit `2` with second digit `0`–`3`. The minute is `0`–`5` then any digit.

$$(\,[01]\,D \;\mid\; 2\,[0\text{-}3]\,)\ \texttt{:}\ [0\text{-}5]\,D$$

Checks. `09:30`: hour case 1 (`0`,`9`), minutes `3`,`0`. `23:59`: hour case 2 (`2`,`3`), minutes `5`,`9`. `24:00`: first digit `2` forces the second into `[0-3]`. `9:30`: the hour needs two digits, no alternative has one. `12:60`: the minute's first digit is limited to `[0-5]`.

Gotcha

$D\,D$`:`$D\,D$ "and check the range later" is a semantic check, not a regex; if the question says *regular expression for valid times*, the ranges go into the alternatives. A numeric range is always written by **splitting on the leading digit**.

6 No two consecutive `b`'s

Practice

Given. Alphabet $\{a, b\}$. The language of all strings (including ε) that do not contain `bb`.

Your task. (a) Write the regex. (b) Show how ε, `b`, `abab`, `bab` match. (c) Explain why `bb` and `abba` do not. (d) Give the three-state DFA.

Solution

Parts. Every b except possibly the last must be followed by an a, so the string is a sequence of blocks a or ba, optionally ending with a lone b.

$$(\,a \;\mid\; ba\,)^*\;(\,b\,)?$$

Checks. ε: zero blocks, no final b. `b`: zero blocks, the final b. `abab`: blocks a, ba, then the final b. `bab`: block ba, final b.
`bb`: the first b must start a block ba (no a follows) or be the final b (but another symbol follows). `abba`: after block a, the first b needs an a next; it gets a b.

DFA. States: 0 "last symbol not b" (start, accepting), 1 "last symbol b" (accepting), dead. $0 \xrightarrow{a} 0$, $0 \xrightarrow{b} 1$, $1 \xrightarrow{a} 0$, $1 \xrightarrow{b}$dead. Three states, as the distinguishable strings ε, b, bb demand.

Gotcha

$(a \mid ba)^*$ alone loses every string ending in b (`b`, `ab`); $(a \mid b)^*$ with "but not `bb`" written in words is not a regex. "Does not contain w" is written by describing what the string *is* made of, never by negation.

7 Comments, a different pair of delimiters

Practice

Given. Alphabet $\{(,), *, x\}$. A Pascal comment opens with `(*`, closes with the **first** `*)`, and is not nested.

Your task. (a) Write the regex (the mock's 2.3(a) with new symbols: do it without looking). (b) Show how `(**)`, `(*x*)`, `(*(*x**)` match. (c) Explain why `(*x*)x*)` is not one comment. (d) Is the nested version regular?

Solution

Parts. Opener `(*`; a body that never contains `*)`: items x, `(`, a lone `)` (harmless: only `*)` closes), or a run of stars followed by something that cannot close, i.e. x or `(`; the closer is a run of one or more stars and `)`.

$$\texttt{(*}\ (\ x\ \mid\ \texttt{(}\ \mid\ \texttt{)}\ \mid\ \texttt{*}^{+}\,(x \mid \texttt{()})^{*}\ \texttt{*}^{+}\ \texttt{)}$$

Checks. `(**)`: opener, empty body, closer `*)`. `(*x*)`: body x, closer. `(*(*x**)`: body `(`, then the item `*`x (a star run followed by x), closer `**)` (the star run of the closer absorbs both stars). `(*x*)x*)`: after `(*x` the symbols `*)` can only be the closer (a star run inside the body must be followed by x or `()`, so the comment ends at `(*x*)`; the rest is outside it.

Nested. No: accepting `(* a (* b *) c *)` as one comment needs the number of open `(*`, and the strings $(\texttt{(*)})^1, (\texttt{(*)})^2, \ldots$ are pairwise distinguishable (by how many `*)` close them), so no finite DFA works. A lexer uses a start condition and a depth counter.

Gotcha

Forgetting that the body may contain a star or a lone `)`: `(*`$(x \mid \texttt{()})^{*}$`*)` rejects `(*x**)`, `(*(*x**)` and `(*)*)`. The mock's alphabet has the same trap with a lone `/`: the second symbol of the closer is harmless on its own, only the pair closes. Forgetting that the closer may have several stars: `...*)` with a body item `*`^{+}x still fails on `(***)`, which needs `*`$^{+}$`)` as the closer.

8 Identifiers that are not the keyword `if`

Practice

Given. Alphabet $\{i, f\}$. The language of nonempty strings that are not exactly `if`.

Your task. (a) Write the regex. (b) Explain why a real lexer never writes this regex, and how it gets the same effect.

Solution

Parts. Split by length. Length 1: $i \mid f$. Length 2, all but `if`: $ii \mid fi \mid ff$. Length ≥ 3: anything.

$$i \ \mid\ f \ \mid\ ii \ \mid\ fi \ \mid\ ff \ \mid\ (i \mid f)(i \mid f)(i \mid f)^{+}$$

Why nobody writes it. Regular languages are closed under complement, so the regex always exists, but it is unreadable and grows with every keyword. A lexer lists the keyword rule `if` *before* the identifier rule $(i \mid f)^{+}$: on `if` both rules match the same length and the tie goes to the first rule; on `iff` only the identifier rule gives the longest match. Mock 2.4 is this question with the two rules swapped (then `if` becomes an identifier and the keyword rule is unreachable).

Gotcha

Writing "$(i \mid f)^{+}$ except `if`" or ¬`if`: not regex syntax. If a question asks for the regex, build it by length or by first-difference; if it asks what the lexer does, answer with rule order and longest match.

9 C integer literals: decimal, octal, hexadecimal

Practice

Given. Alphabet $D \cup \{\texttt{x}, a, \ldots, f\}$. A C integer literal is a decimal number with no leading zero (`0` itself is allowed), or an octal number (`0` followed by one or more digits `0`–`7`), or a hexadecimal number (`0x` followed by one or more hex digits).

Your task. (a) Write the regex. (b) Show the matches of `0`, `42`, `017`, `0xff`. (c) Explain why `09`, `0x`, `x1` are not matched.

Solution

Parts. Four alternatives, one per shape; the leading-zero rule splits "decimal" into `0` alone and "nonzero digit then digits".

$$0 \;\mid\; [\texttt{1-9}]\, D^* \;\mid\; 0\,[\texttt{0-7}]^+ \;\mid\; 0\,\texttt{x}\,[\texttt{0-9}a\texttt{-}f]^+$$

Checks. `0`: first alternative. `42`: nonzero `4`, then `2`. `017`: octal, `0` then `17`. `0xff`: hex, `0x` then `ff`.
`09`: decimal forbids a leading zero, octal forbids the digit `9`, hex needs `x`. `0x`: the hex digits are $^+$, mandatory. `x1`: every alternative starts with a digit.

Gotcha

$D^+ \mid 0\texttt{x}H^+$ accepts `09` and gives `017` the wrong meaning. The lexer does not care which alternative matched; the *value* (octal 017 = 15) is computed in the action, so if a question asks for separate tokens DEC, OCT, HEX you need three rules, and `0` must go to exactly one of them (put it in DEC, or make octal $0[\texttt{0-7}]^*$ and drop it from decimal).

10 Line comments

Practice

Given. Alphabet $\{\texttt{/}, x, \texttt{\textbackslash n}\}$. A line comment starts with `//` and runs to the end of the line; the newline is **not** part of the comment.

Your task. (a) Write the regex. (b) What does a longest-match scanner return on `//x/x\nx`? (c) Why does $\texttt{//}(x \mid \texttt{/} \mid \texttt{\textbackslash n})^*$ break the lexer?

Solution

Parts. Opener `//`; body: anything except the newline, zero or more. No closer.

$$\texttt{//}\ (x \mid \texttt{/})^*$$

(b) The DFA runs through `//x/x`, stops at `\n` (no transition), and the last accepting position is just before it: the token is `//x/x`; the scanner then reads the newline as whitespace and continues with `x`.

(c) With the newline in the body the comment never ends: longest match swallows every following line up to the end of the file, the same way the mock's `.*` swallowed everything up to the *last* `*/`.

Gotcha

"Runs to the end of the line" means the body excludes the terminator; the terminator is left for the next token. A body `(x|/)*` is also why `////` is one comment, not two: the opener is read once and the rest is body.

11 HTML comments: a three-symbol closer

Practice

Given. Alphabet $\{\texttt{-}, \texttt{>}, x\}$. An HTML comment opens with `<!--` (treat `<!` as one fixed opener symbol) and closes with the first `-->`.

Your task. (a) Write the regex with the method box, drawing the states first. (b) Show how `<!---->`, `<!--x->x-->`, `<!--x--->` match. (c) Why is `<!-->` not a comment? (d) Why is `<!--x-->x-->` not one comment?

Solution

States of the body. N (last symbol not a dash), D1 (just read one dash), D2 (just read two or more dashes). From N: x and `>` stay, `-` goes to D1. From D1: `-` goes to D2, x and `>` return to N. From D2: `-` stays, x returns to N, `>` is forbidden.

Loops at N. Stay (x, `>`); one dash then back (`-`(x | `>`)); two or more dashes then back (`--`$^+$ x: only x may follow, a `>` would close). Closer: two or more dashes and `>`.

$$\texttt{<!--}\ (\ x \mid \texttt{>} \mid \texttt{-}(x \mid \texttt{>}) \mid \texttt{--}^+ x\ )^*\ \texttt{--}^+\texttt{>}$$

Checks. `<!---->`: empty body, closer `-->`. `<!--x->x-->`: body x, `->` (one dash then `>`), x; closer. `<!--x--->`: body x; the closer's `--`$^+$ takes all three dashes.

(c) `<!-->`: after the opener only `>` remains; the closer needs two dashes of its own, and the opener's dashes are already spent.

(d) `<!--x-->x-->`: after the body x the symbols `-->` can only be the closer (two dashes then `>` is forbidden inside the body), so the comment ends there; `x-->` is outside.

Gotcha

Writing the body with the two-state recipe, $(x \mid \texttt{>} \mid \texttt{-}^{+}(x \mid \texttt{>}))^*$, accepts `-->` inside the body (`--` followed by `>`). Each extra symbol of the closer is one more state, and the symbol that is forbidden only appears in the last state.

12 An even number of a's

Practice

Given. Alphabet $\{a, b\}$. All strings with an even number of a's (ε included).

Your task. (a) Write the regex. (b) Show how ε, `bb`, `aba`, `babab` match. (c) Explain why `a` and `baaba` do not. (d) What is wrong with $(aa \mid b)^*$?

Solution

Parts. Two states: E (even so far, accepting, start) and O (odd). A b keeps the state; an a flips it. Loops at E: a b, or an a then any number of b's then an a.

$$(\, b \;\mid\; a\, b^*\, a\,)^*$$

Checks. ε: zero loops. `bb`: two b loops. `aba`: one loop $a\,b\,a$. `babab`: b, then the loop $a\,b\,a$, then b.

(c) `a`: the loop $a\,b^*\,a$ needs a second a. `baaba`: b, $a\,a$, b, then a lone a: three a's, odd.

(d) $(aa \mid b)^*$ requires the two a's of a pair to be adjacent: it rejects `aba`, which has an even count. The b^* between the two a's is the whole point.

Gotcha

"Even number of a" is a counting-mod-2 property, regular because two states suffice; "the same number of a and b" is not regular (unbounded count). When you see $(aa \mid b)^*$ in an answer, test it on `aba`.

13 Binary numerals, no leading zeros, multiples of four

Practice

Given. Alphabet $\{0, 1\}$. Binary numerals without leading zeros (`0` is the only numeral that starts with `0`) whose value is a multiple of 4.

Your task. (a) Write the regex. (b) Show how `0`, `100`, `1100` match. (c) Explain why `00`, `0100`, `10`, `1` are not matched.

Solution

Parts. A multiple of 4 in binary ends in `00`, or is `0`. No leading zeros: either `0` alone or `1` followed by anything.

$$0 \;\mid\; 1\,(0 \mid 1)^*\,0\,0$$

Checks. `0`: first alternative. `100`: `1`, empty middle, `00`. `1100`: `1`, middle `1`, `00`.
`00` and `0100`: a numeral starting with `0` must be `0` alone. `10`: only one trailing zero (value 2). `1`: no trailing zeros (value 1).

Gotcha

$(0 \mid 1)^*00$ accepts `00` and `0100`: the leading-zero rule and the divisibility rule are two separate parts and both must appear. For "multiple of 3" no suffix test exists: you need the 3-state remainder DFA and state elimination.

14 Several keywords in one regex, and the identifier rule

Practice

Given. Alphabet $\{i, f, n, t, e\}$. The keywords are `if`, `in`, `int`; identifiers are $(i \mid f \mid n \mid t \mid e)^+$.

Your task. (a) Write one regex for the three keywords, sharing their common prefixes. (b) With the keyword rule listed before the identifier rule, what does the scanner return on `in`, `inte`, `i`? (c) Same question with the identifier rule listed first.

Solution

(a) Factor the common prefix `i`, then `n`:

$$i\;(\;f \;\mid\; n\,(t)?\;)$$

(b) `in`: both rules match two characters, tie, first rule wins: KEYWORD in. `inte`: keyword matches at most three characters (`int`), identifier matches four: longest match, ID inte. `i`: the keyword regex has no one-character match; ID i.

(c) Identifier first: `in` is a tie again, now won by the identifier: ID in; the keyword rule can never win a tie, so it is unreachable (flex warns). `inte` and `i` unchanged.

Gotcha

Longest match is decided *before* rule order: order only breaks ties of equal length. `inte` is an identifier under both orders; nobody "backs up" to the keyword `int` because a longer match exists.

15 Exactly one occurrence of `ab`

Practice

Given. Alphabet $\{a, b\}$. All strings in which `ab` occurs exactly once.

Your task. (a) Write the regex. (b) Show how `ab`, `aab`, `abb`, `babaa` match. (c) Explain why ε, `ba`, `abab` are not matched.

Solution

Parts. Cut the string at its only `ab`. The part before must not contain `ab`: a string with no `ab` is b^*a^* (once an a appears no b may follow). Same for the part after. Gluing $b^*a^* \cdot ab \cdot b^*a^*$ and merging the runs:

$$b^*\ a^+\ b^+\ a^*$$

Checks. `ab`: $a^+ = a$, $b^+ = b$. `aab`: $a^+ = aa$. `abb`: $b^+ = bb$. `babaa`: $b^* = b$, $a^+ = a$, $b^+ = b$, $a^* = aa$; the only a followed by b is the second symbol.
ε: a^+ and b^+ are mandatory. `ba`: b, a, then b^+ is missing. `abab`: after $a\,b\,a$ the trailing a^* cannot take a b: a second `ab` would start.

Gotcha

"Exactly one" and "at most one" are regular because you can describe the pieces between the occurrences; the piece "contains no `ab`" is b^*a^*, not $(a \mid b)^*$ with a wish attached. "The same number of `ab` and `ba`" would still be regular (they alternate), but "as many a as b" is not.

16 Amounts with thousands separators

Practice

Given. Alphabet $D \cup \{$`,`, `.`, `-`$\}$. An amount is an optional minus sign, then an integer part written with a comma every three digits from the right (`0`, `999`, `1,000`, `12,345,678`; no leading zero except `0` itself), then an optional fraction: a dot and one or more digits.

Your task. (a) Write the regex. (b) Show how `0`, `-1,000`, `12,345.67` match. (c) Explain why `1000`, `01`, `1,00`, `1,0000`, `1,000.` are not matched.

Solution

Parts. Sign, optional. Integer part: `0`, or a first group of one to three digits with a nonzero first digit, then zero or more groups of exactly `,`DDD. Fraction, optional, `.`D^+.

$$(\text{-})?\ \Big(\ 0\ \mid\ [\text{1-9}]\,(D\,(D)?)?\ (\ \text{,}\ D\,D\,D\ )^*\ \Big)\ (\ \text{.}\ D^+\ )?$$

Checks. `0`: no sign, the `0` alternative, no fraction. `-1,000`: sign, first group `1`, one group `,000`. `12,345.67`: first group `12`, group `,345`, fraction `.67`.
`1000`: the first group holds at most three digits and the fourth must come after a comma. `01`: the `0` alternative is `0` alone, the other needs a nonzero first digit. `1,00`: a group is exactly three digits. `1,0000`: same, four. `1,000.`: the fraction's D^+ is mandatory.

Gotcha

"One to three digits" is written [1-9](*D*(*D*)?)? with the core constructs; `D{1,3}` is flex sugar and is wrong anyway here because the first digit is restricted. "Every three digits" is a repeated group `,`*DDD* attached to the *end*, like the digit separators of §1.